Flow-Matching Action Expert: Diagnostic Report

Yaak · `rmind` · branch `feat/action_expert` · 2026-06-08

Abstract. We diagnose a flow-matching policy head that predicts a 6-step steering/throttle chunk on top of a frozen pretrained driving encoder. The motivating question is whether a generative (flow) head is worth its complexity over a deterministic regression head, and if so what currently limits it. A synthetic oracle shows the decoder itself is not the bottleneck (it reaches L1 ≈ 0.017 on a clean task). On real data we find four things. (1) The deployed Euler/8 ODE sampler was leaving ≈ 0.015 L1 of pure integration error on the table; Heun/32 removes it, and the apparent superiority of **coarser** Euler is an artifact of variance collapse, not accuracy. (2) Enriching the conditioning with 256 image tokens does nothing — the decoder substitutes information sources to the same error floor, so conditioning content is not the limiter. (3) The 6-step chunk is **constant in expectation**: the model predicts one well-anticipated action and copies it across slots, because the slot position embeddings never receive training signal. (4) Infrastructure bugs, most importantly an "overfit" configuration that silently trained on the full corpus, which reframes several earlier conclusions. We close with the experiment that now gates everything.

1 Background and question

A driving policy must map the current scene to future controls. The standard choice is a deterministic head trained with MSE — fast, stable, but it **regresses to the mean**: when the data contains more than one reasonable action (e.g. "brake" vs "swerve", or simply human steering jitter), MSE returns their average, which can be a control neither human would take. A **flow-matching** head instead learns the full conditional distribution of actions: it trains a velocity field $v(x, t \mid c)$ and, at inference, integrates a sample from noise ($t = 0$) to an action ($t = 1$). The promised payoff is that it can represent multiple modes and commit to one, rather than averaging them.

That promise is only worth the added machinery (an ODE solver at inference, a noisier training objective) if (a) the data is actually multimodal at the relevant granularity and (b) the head can be made accurate enough. This report is a systematic attempt to locate where the current implementation stands on both counts, and to separate genuine modelling limits from incidental bugs.

The head (`FlowPolicyObjective` + `FlowActionDecoder`) is a small cross-attention transformer (dim 256, 4 layers) conditioned on summary and waypoint tokens from a frozen encoder, trained with an MSE flow-matching loss and logit-normal flow-time sampling. The encoder is frozen, so every result here concerns the **head**, not the representation.

2 Method: oracle ceiling and per-frame diagnostics

To know whether a given error is the decoder's fault or the task's, we first run a **synthetic oracle**: the same decoder overfitting a single batch of random condition→action pairs. This isolates raw capacity. It collapses the task to L1 ≈ 0.017 and shows depth 2→4 helps but width and extra solver steps do not (Table 1). Conclusion: the decoder and sampler are sound; any larger real-data error lives in conditioning, objective, or data — not the architecture.

config	flow_mse	sample_l1	note
dim 32, 2L	0.116	0.102	under-capacity
dim 256, 2L	0.010	0.021	old default
dim 256, **4L**	0.007	**0.017**	best; chosen
dim 384, 2L	0.011	0.023	width: no gain
Heun 8→32 steps	—	0.026→0.022	solver steps: negligible (oracle)
uniform vs logit-normal t	—	0.030 vs 0.017	logit-normal wins

Table 1: Synthetic oracle (overfit one batch). The decoder reaches 0.017; capacity is tapped past 4 layers / dim 256.

On real data we evaluate per frame by drawing $N = 32$ action samples and asking how they cluster around the logged action. Two readouts recur: **hit@ε**, the fraction of the 32 draws landing within ε of ground-truth first-step steering (a concentration measure), and **best-of-32 L1** (`bo32`), the error of the single closest draw (a coverage measure — does the distribution contain the right answer at all?). We separate **spike** frames (|steering| > 0.5, i.e. active maneuvers, 14% of frames) from **flat** frames, because drive-wide averages are dominated by straight-road driving and hide exactly the moments that matter.

3 Integration error, and why coarser Euler looks better

Sampling integrates the learned ODE; the solver and its step count (jointly, the number of function evaluations, NFE) trade compute for integration accuracy. The deployed default was Euler with 8 steps. Sweeping solver and steps on a fixed checkpoint (table below) shows the **Heun** family converging to a plateau at flat L1 $\approx$ 0.040 — this plateau is the field's true exact-flow quality, and the old Euler/8 setting sat well above it (0.058), i.e. 0.015 L1 was pure solver error, not model error.

The surprise is that **coarser** Euler scores a higher hit-rate (Euler/16 reaches 89% flat hit@.05). This is not better accuracy: its `bo32` simultaneously gets **worse** (2.9 vs 2.4 $\times 10^{-3}$). A coarse step under-resolves the sharpening the flow does near $t = 1$ and contracts the draws toward the conditional mean — trading away the sample diversity that is the entire point of a generative head, to win a unimodal single-draw metric. In the limit this is just a 1-step mean regressor, i.e. the Gaussian baseline. That midpoint/16 $\approx$ Heun/16 (not Euler/16) rules out an endpoint-evaluation explanation and pins the cause on step coarseness itself.

sampler	NFE	flat hit@.05	flat L1	flat bo32	spike hit@.05
Heun/8 (old)	16	53%	.058	3.1e-3	34%
Euler/16	16	**89%**	**.025**	2.9e-3	**49%**
Heun/16	32	72%	.041	2.7e-3	42%
midpoint/16	32	69%	.043	2.7e-3	40%
Euler/32	32	81%	.034	2.4e-3	46%
Heun/32	64	73%	.040	2.4e-3	42%
Euler/64	64	78%	.036	2.5e-3	44%
Heun/128	256	74%	.039	2.4e-3	42%

Table 2: Integrator sweep, one checkpoint. Heun plateau ($\approx$0.040, bo32 0.0024) is the honest field quality; Euler "wins" only by contraction.

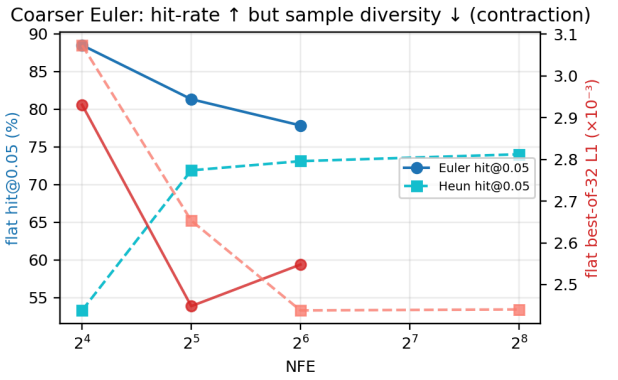

Figure 1: As NFE drops, Euler hit-rate rises (blue) while best-of-32 error rises too (red): diversity is being collapsed, not error reduced.

Decisions. Report and validate at **Heun/32** — it measures the field, not the solver. The **deployment** sampler is deliberately left open: coarse-Euler's contraction helps on this unimodal overfit but would erase flow's advantage on genuinely multimodal data, so it must be chosen against closed-loop driving, not this metric. Note the oracle's "more steps don't help" did **not** transfer: that was Heun on a smooth synthetic field, whereas the real learned field is curvier and needs the steps.

4 Conditioning content is not the bottleneck

The head conditions on compressed summary tokens. A natural hypothesis was that this is too thin and that giving the decoder the raw scene would help, so we added the 256 image patch tokens to the condition. It changed nothing (Table 3, rows 1–2 identical within noise). To check this was not because the tokens were ignored, we took the **image-trained** checkpoint and removed the image tokens at inference: it collapsed completely (flat L1 0.86). So the decoder does use them — it simply **substitutes** whatever conditioning it is given and lands at the same floor either way. The limiter is therefore not the information available to the head; adding conditioning channels is not where to invest.

conditioning (matched Heun/8)	flat hit@.05	flat L1	spike L1
summaries + waypoints	54.8%	.055	.130
1. 256 image tokens	53.3%	.058	.132
image ckpt, image **ablated**	1.9%	.859	.944

Table 3: Conditioning is a null. Rows 1–2: adding image tokens does nothing. Row 3: yet the image checkpoint genuinely depends on them — it substitutes sources rather than accumulating information.

5 Training: averaging weights does not help; training longer does

Two training-side levers were tested. **Weight EMA** (an exponential moving average of parameters, standard in diffusion work to smooth a noisy optimizer) was consistently 2 points **worse** here (Table 4). The reason is diagnostic: EMA helps when weights orbit an optimum under noisy gradients, but on this tiny overfit the weights are in slow steady descent, so the trailing average simply lags behind. **Training duration**, by contrast, is a real lever: 100→400 epochs nearly halves flat L1. Earlier runs appeared to plateau around epoch 40, but that

"plateau" was the Heun/8 integration error (§3) masking continued improvement underneath — once measured cleanly, the field keeps getting better.

checkpoint (Heun/32)	flat hit@.05	flat L1	spike hit@.05	spike L1
100 ep (legacy)	54.8%	.055	35.8%	.130
raw, 400 ep	**86.3%**	**.028**	**55.6%**	**.096**
EMA, 400 ep	84.6%	.030	52.2%	.100

Table 4: Training duration dominates EMA. EMA is retained for the eventual multi-drive regime (where gradients are genuinely noisy) but is a null here.

6 What the chunk actually predicts

A 6-step action chunk should describe how the control **evolves** over the next six steps. We tested this by aligning each slot's prediction to the frame it is meant to predict. A pure "copycat" (predicting the current action for all six steps) would show error growing sharply with horizon; the model does **not** do that — its per-slot error is flat, even slightly U-shaped (Figure 2, left). But neither does it predict six distinct steps. Each slot's prediction best matches the target of slot ≈ 3 (alignment shift $= 3 - h$, perfectly linear, Figure 2 right), and the mean chunk's internal slope has **zero** correlation with the ground-truth slope. In other words the model predicts **one** action — well anticipated, about three steps into the future — and writes that same value into all six slots.

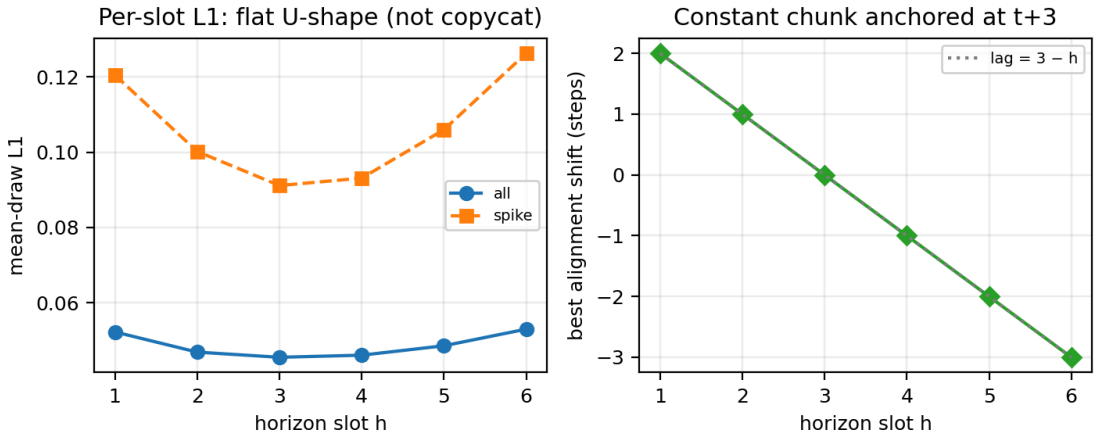

Figure 2: Left: per-slot L1 is flat/U-shaped — not the rising staircase of a copycat. Right: each slot aligns to a target 3−h steps away, i.e. all slots are the same mid-horizon (t+3) estimate.

The cause is mechanical and was confirmed by inspecting the trained weights: the slot **position embeddings** — the only input that distinguishes one slot from another — never moved from their random initialization (row-norm 0.32 = init, mutual cosine ≈ 0 after 100 epochs), whereas the zero-initialized time modulation in the same blocks trained normally ($\|W\| \approx 0.8$). The within-chunk structure is a vanishingly small fraction (4×10^{-4}) of the MSE loss, so there was never gradient pressure to use the slot identity; a constant chunk is the loss-optimal solution. This is an **incentive** problem, not a wiring fault.

Fixes implemented (awaiting the corrected overfit to validate): a within-chunk **delta loss** that supervises differences between adjacent slots — zero on flat stretches, pure phase signal at maneuvers, and impossible to satisfy with a constant chunk; rotary position encoding in the slot attention (a complement to, not a replacement for, the learned embedding, since rotation alone cannot distinguish identical slot contents); and two probes, `pe_drift` and a 6×6 slot similarity image, that report directly whether the slots are learning to differ.

7 Flow versus the regression baseline

The case for keeping the flow head, despite the above, is visible at maneuvers (Figure 3). The 32 draws there are not scattered noise: their **mass** sits on the ground-truth steering extreme ($\approx$20/32 within 0.1, reaching full ±1). A deterministic MSE head, on the same maneuvers, undershoots — pulled toward the mean of the turn. The distinction matters for the roadmap: the flow head's current weakness (samples too spread) is **reducible** by the levers above (solver, duration, field collapse), whereas the regression head's weakness (structural undershoot of large actions) is not. That is the argument for continuing with flow, to be confirmed in closed loop.

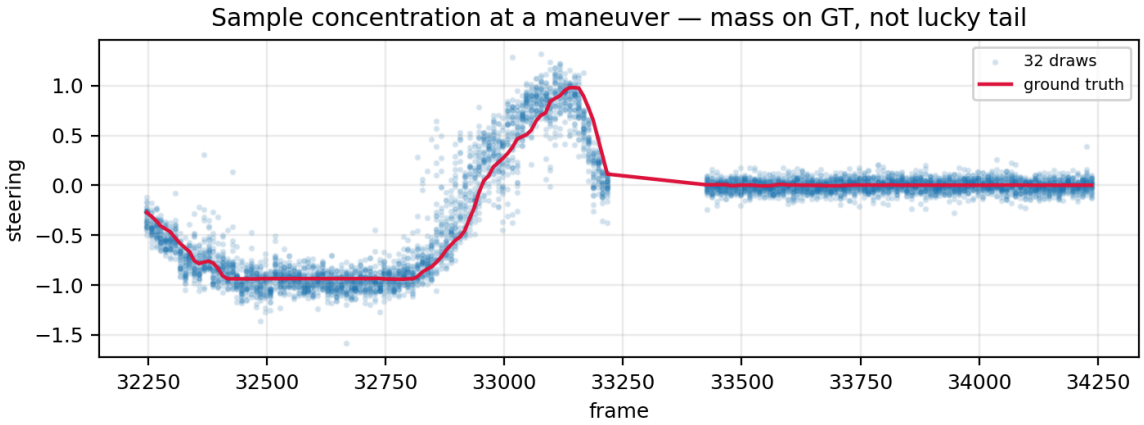

Figure 3: 32 draws per frame across a maneuver. The distribution covers the GT trajectory to full magnitude — coverage, not a lucky best-of-N tail.

8 Infrastructure bugs found and fixed

Several of the above conclusions were nearly derailed by infrastructure issues; they are listed because each is a recurring trap, and one materially reframes the rest.

bug	**resolution / consequence**
The "overfit" datamodule borrowed the full `yaak/train` dataset (4586 drives), so every "overfit" run actually trained on the whole corpus and was only **evaluated** on one drive.	Repoint to the single-drive dataset. Reframes the "underfit floor" as generalization difficulty — **a true single-drive overfit has not yet been run**, and is now the gating experiment.
Predicted-action plot showed a sawtooth with period = batch size; the RNG was reseeded per batch, making the noise a function of within-batch index.	Sample with the global RNG; reproducibility via `seed_everything`.
A non-finite condition field (waypoints/Gnss, not range-checked by the data filter) produced NaN losses once training was confined to one drive.	Drop non-finite rows in the loss (per-sample safe) and name the offending field; a data-layer filter is the permanent fix.
The new delta loss with `action_horizon=1` differenced a length-1 axis → empty tensor → NaN every step.	Auto-disable the delta term for horizon < 2 (it is undefined there).
Two runs sharing one dataset cache crashed each other.	Use an isolated cache path per concurrent run.

Table 5: Infrastructure bugs. The datamodule misconfiguration is the most consequential, as it changes how the overfit numbers should be read.

9 Conclusions and next steps

The decoder and sampler are not the problem (oracle 0.017), and neither is the conditioning content (image-token null). What remains are a measurement fix (Heun/32, now adopted), a clear structural defect (the chunk does not differentiate its slots, because nothing rewards it — addressed by the delta loss and RoPE), and the realization that we had never actually run the single-drive overfit we thought we were running.

That overfit is the immediate priority: run it on the corrected single-drive datamodule and ask whether the field reaches the oracle's 0.017 when genuinely asked to memorize. It doubles as the first clean test of the delta loss, RoPE, and EMA together — watch `pe_drift` lift off zero and the horizon lag flatten. The questions that need real multi-drive data — genuine multimodality (best-of-N gain), EMA under noisy gradients, the deployment sampler — wait for that setting, where flow's premise can finally be tested rather than assumed.